

High-Performance Fine-Tuning of BERT for Question Answering on SQuADv2

Devanshi Bhavsar
New York University
dnb7638@nyu.edu

Nikhil Arora
New York University
na4063@nyu.edu

Abstract—We study the efficiency-accuracy trade-offs of fine-tuning `bert-base-uncased` for extractive question answering on SQuAD v2 using a single NVIDIA T4 GPU. Starting from the Hugging Face `run_qa.py` script, we design a grid of configurations that vary system-level kernels (SDPA, `torch.compile`, dynamic padding/bucketing), parameter-efficient fine-tuning (LoRA, BitFit, layer freezing), and memory-oriented techniques (8-bit optimizers, gradient checkpointing, AOT-eager backend). Our experiments show that modern compiler and attention kernels can deliver up to $\approx 17\%$ higher training throughput over a strong FP16 baseline on T4 while slightly improving F1, and that 8-bit optimizers and gradient checkpointing can reduce peak VRAM by up to $\approx 30\%$ with minimal accuracy loss. In contrast, LoRA and BitFit give substantial speedups but significantly degrade SQuAD v2 F1 under our training budget, suggesting that aggressive parameter efficiency is less effective for extractive QA in this small-model, low-resource regime. We summarize the results as practical “recipes” for training BERT on commodity GPUs.

Index Terms—BERT, SQuAD v2, question answering, high-performance machine learning, PyTorch, SDPA, `torch.compile`, LoRA, BitFit, 8-bit optimizer, gradient checkpointing

I. INTRODUCTION

A. Background and Motivation

Transformer-based language models such as BERT have become the standard backbone for extractive question answering tasks. However, fine-tuning even a relatively small model like `bert-base-uncased` can be slow and memory-intensive on commodity GPUs such as NVIDIA T4 (16 GB). Many academic and industrial results assume A100-class hardware, leaving a gap for realistic training recipes on smaller accelerators.

B. Problem Statement

Our goal in this project is to answer the following question: *How much hardware-level efficiency can we unlock in BERT-base fine-tuning on SQuAD v2 on a single T4 GPU using modern compiler and attention kernels, and do parameter-efficient methods still help once kernels are fused and optimized?*

Concretely, we focus on: (1) measuring the impact of SDPA, `torch.compile`, and dynamic padding/bucketing on throughput and accuracy; (2) comparing parameter-efficient fine-tuning (LoRA, BitFit, layer freezing) against full fine-tuning under a fixed training budget; and (3) characterizing memory-oriented techniques such as 8-bit optimizers, gradient checkpointing, and AOT-eager.

C. Objectives and Scope

We restrict ourselves to a single encoder model (BERT-base) and a single extractive QA dataset (SQuAD v2), running on T4 GPUs in the NYU HPC cluster. Within this scope, our objectives are:

- establish a strong FP16 baseline for BERT-base on SQuAD v2;
- incrementally add system optimizations such as SDPA, `torch.compile` and dynamic padding;
- compare LoRA, BitFit, and layer freezing to full fine-tuning in terms of accuracy and throughput;
- study memory trade-offs of 8-bit AdamW, gradient checkpointing, and AOT-eager; and
- summarize these results as practical “recipes” for training BERT on commodity GPUs.

The rest of the paper is organized as follows: Section II reviews relevant work, Section III describes our methodology, Section IV presents experimental results, Section V discusses implications and limitations, and Section VI concludes.

II. LITERATURE REVIEW

LoRA [1] introduces low-rank adapters that enable parameter-efficient fine-tuning by updating small rank- r matrices instead of the full weight matrices. BitFit [2] proposes tuning only bias terms with surprisingly small accuracy gaps to full fine-tuning on several NLP benchmarks. FlashAttention and its successors [3] show that I/O-aware fused attention kernels can significantly increase attention throughput on modern GPUs. Dettmers et al. [4] demonstrate that 8-bit optimizer states can dramatically reduce memory footprint while maintaining competitive accuracy. For SQuAD v2 specifically, stronger encoder architectures such as DeBERTa-v3 [5] report dev F1 scores above 90, but on larger models and higher-end hardware.

At a high level, prior work tends to study parameter-efficient methods or kernel-level accelerations in isolation, often on A100/V100 hardware and larger models. Our work differs in that we treat these techniques as components in a single design space targeted at a constrained T4 setting, and we report end-to-end trade-offs (accuracy, throughput, VRAM) for a fixed model and dataset. The goal is not to push absolute SQuAD v2 accuracy, but to provide empirically grounded guidelines for practitioners fine-tuning BERT-base on commodity GPUs.

III. METHODOLOGY

A. Dataset and Preprocessing

We use the SQuAD v2 dataset via the Hugging Face datasets library. All runs enable the unanswerable questions setting (version_2_with_negative). We follow the Hugging Face run_qa.py pipeline:

- **Tokenizer:** BertTokenizerFast with max_seq_length = 384 and doc_stride = 128.
- **Task:** extractive QA, predicting start and end token indices or “no answer”.
- **Splits:** train for fine-tuning; validation for all reported metrics.

B. Model and Baseline Setup

Our base model is bert-base-uncased with the standard QA head (BertForQuestionAnswering), approximately 109M parameters. Unless otherwise noted, all experiments use:

- batch size 8 (train) and 16 (eval),
- 3 training epochs,
- learning rate 3×10^{-5} for full fine-tuning,
- AdamW optimizer,
- FP16 mixed-precision training, and
- Hugging Face Trainer and run_qa.py script as the base.

The **A1** configuration is our reference baseline: standard BERT attention, padding to a fixed length, no SDPA, and no torch.compile.

C. Configuration Grid

To isolate the effect of each technique we define four families of configurations:

1) A-series: System Optimizations:

- **A1:** Baseline (full fine-tuning, standard attention, no compiler).
- **A2:** A1 + PyTorch SDPA attention.
- **A3:** A2 + torch.compile with the inductor backend.
- **A4-nb:** A3 + dynamic padding within each batch (no bucketing).
- **A4-b:** A3 + length-based bucketing (intended to combine dynamic padding and fewer padding tokens, but found to be buggy).

2) B-series: *Parameter-Efficient Fine-Tuning:* These configurations start from the A3 baseline (SDPA + torch.compile) and modify which parameters are trainable:

- **B1: LoRA** — low-rank adapters on attention projections; only LoRA weights are updated.
- **B2: BitFit** — only bias terms are updated.
- **B3: Layer Freezing** — encoder layers are partially frozen; only a subset of later layers and the QA head are trainable.

3) C-series and D1: Memory and Backend:

- **C1: 8-bit AdamW** — A3 with an 8-bit optimizer (via bitsandbytes) to compress optimizer states.
- **C2: Gradient Checkpointing** — A3 with gradient checkpointing enabled to reduce activation memory.
- **D1: AOT-Eager** — A3 with torch.compile backend switched from inductor to aot_eager.

Throughout this work we focus on *training* efficiency (throughput and VRAM during fine-tuning); we do not optimize or benchmark inference latency explicitly.

D. Profiling and VRAM Measurement

We use two kinds of instrumentation:

- **Throughput and runtime** are taken from Hugging Face Trainer logs (samples/s and steps/s during training, total train runtime).
- **VRAM usage** for A3, C1 and C2 is measured on a short run (single warmup step followed by one measured step) using torch.cuda.max_memory_allocated() and max_memory_reserved().

Due to cluster restrictions we were unable to use Nsight Systems or Nsight Compute, so we rely on PyTorch-level profiling and aggregated metrics instead of detailed kernel traces.

E. Evaluation Metrics

We report:

- **Exact Match (EM)** and **F1** on the SQuAD v2 validation set,
- **training throughput** in samples/second and steps/second, and
- **peak VRAM usage** (MiB) for selected configurations.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

All experiments run on NYU HPC nodes with a single NVIDIA T4 GPU (16 GB), PyTorch 2.x, and CUDA-enabled mixed precision. We keep the data pipeline, optimizer, and training schedule fixed across runs, varying *one main factor* at a time as described above.

B. Configuration Summary

Table I summarizes the main configurations.

C. Accuracy and Throughput

Table II reports dev F1 and training throughput for the core configurations (numbers rounded).

Key observations from Tables II and III are:

- SDPA and torch.compile (A2, A3) provide modest but consistent throughput gains while keeping F1 essentially unchanged.
- Dynamic padding without bucketing (A4-nb) yields the best overall trade-off: $\approx 17\%$ higher throughput than A1 with slightly higher F1.

TABLE I
SUMMARY OF EXPERIMENTAL CONFIGURATIONS.

ID	Description
A1	Baseline: full FT, std. attention, no compile
A2	A1 + SDPA attention
A3	A2 + <code>torch.compile</code> (inductor)
A4-nb	A3 + dynamic padding (no bucketing)
A4-b	A3 + bucketing (buggy ablation)
B1	A3 + LoRA adapters on attention
B2	A3 + BitFit (bias-only)
B3	A3 + freeze most encoder layers
C1	A3 + 8-bit AdamW optimizer
C2	A3 + gradient checkpointing
D1	A3 with <code>aot_eager</code> backend

TABLE II
SQUAD v2 DEV F1 AND TRAINING THROUGHPUT ON T4.

Config	Dev F1	Train samples/s
A1	76.6	43.9
A2	76.6	42.0
A3	76.9	45.3
A4-nb	77.2	51.6
A4-b	26.9	68.5
B1	54.9	70.4
B2	58.0	73.1
B3	72.0	89.8
C1	76.3	46.4
C2	74.5	37.1
D1	76.4	45.5

- The bucketing variant (A4-b) achieves the fastest throughput but catastrophically fails on accuracy, confirming a bug in our early bucketing implementation.
- LoRA (B1) and BitFit (B2) offer good speedups but suffer large drops in F1 under our 3-epoch budget.
- Freezing most encoder layers (B3) runs fastest and retains acceptable F1 (around 4 points below A3).
- 8-bit AdamW (C1) preserves A3-level F1 with small overhead; gradient checkpointing (C2) saves memory but reduces both throughput and F1.
- AOT-eager (D1) behaves similarly to inductor on T4 for this workload.

D. Memory Usage

Table III summarizes peak allocated VRAM for A3, C1, and C2 measured on a short training run (one warmup step plus one measured step).

TABLE III
SINGLE-STEP PEAK VRAM USAGE ON T4 (FP16).

Config	Peak allocated VRAM (MiB)
A3 (SDPA + compile)	≈ 3087
C1 (8-bit AdamW)	≈ 2466
C2 (grad checkpointing)	≈ 2100

These numbers suggest that 8-bit optimizers reduce peak VRAM by roughly 20% relative to A3, while gradient checkpointing achieves a reduction of about 30%, at the cost of additional compute.

E. Figures

In the final version we include:

- **Fig. 1:** Bar chart of training throughput for all configurations (A1–D1).
- **Fig. 2:** Bar chart of SQuAD v2 dev F1 for all configurations (A1–D1).
- **Fig. 3:** Bar chart of peak VRAM for A3, C1, and C2.

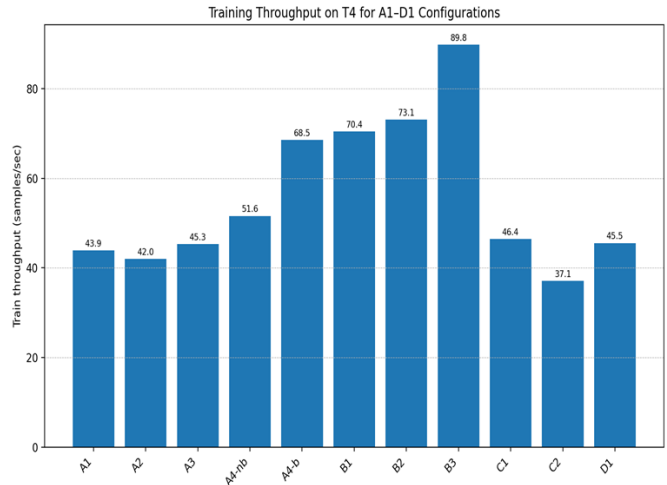


Fig. 1. Training throughput on T4 for all configurations.

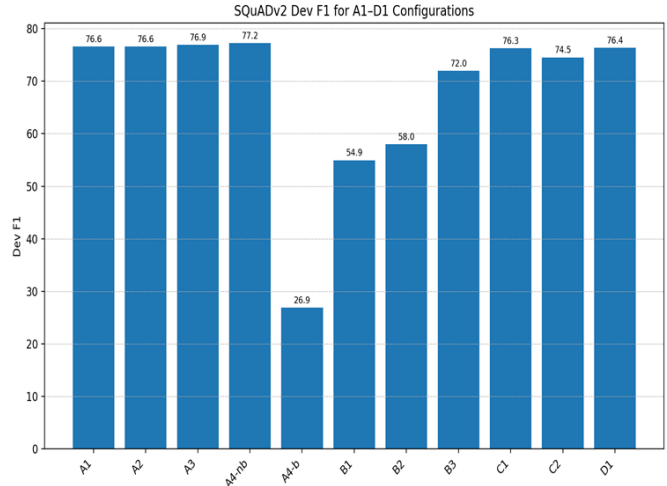


Fig. 2. SQuAD v2 dev F1 for all configurations.

V. DISCUSSION

A. Interpretation of Results

The A-series experiments confirm that on a T4 GPU, modern attention and compiler kernels are beneficial but not transformative by themselves. SDPA changes the attention implementation but does not radically alter end-to-end throughput because the model is still relatively small. `torch.compile`

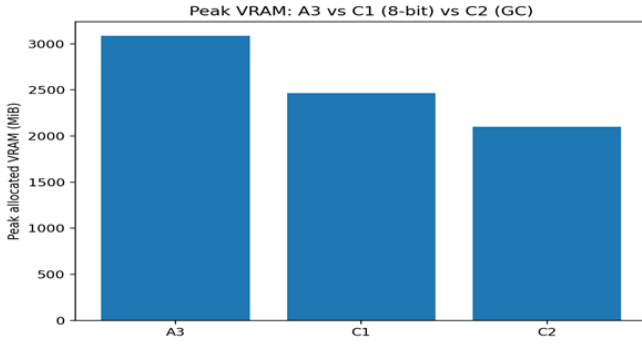


Fig. 3. Single-step peak VRAM on T4 for A3 (baseline), C1 (8-bit AdamW) and C2 (gradient checkpointing).

reduces Python overhead and fuses kernels, leading to a few percent speedup.

Dynamic padding without bucketing (A4-nb) is the most effective system optimization we implemented. By trimming unused padding tokens at the batch level, it increases effective tokens per second without changing the model.

The A4-b run illustrates a common pitfall in dynamic batching: if we reorder or group examples without updating their labels consistently, the model can converge to degenerate solutions while still appearing to train normally. We explicitly treat A4-b as a failed ablation, not as an efficiency win.

Parameter-efficient methods show a trade-off that is unfavorable in our setting. LoRA and BitFit greatly reduce the number of trainable parameters and can speed up training, but with only 3 epochs on SQuAD v2 and a relatively small base model, their F1 lags far behind full fine-tuning. Layer freezing (B3) offers a more balanced compromise.

B. Ablation: A4-b Bucketing Failure

Our A4-b configuration was intended to combine SDPA, `torch.compile`, and length-based bucketing for further efficiency gains. In practice it achieved the highest throughput but catastrophically low F1 (around 26.9), unlike A4-nb, which uses dynamic padding without bucketing and reaches 77.2 F1.

The key issue is label alignment. SQuAD v2 answers are defined as start and end character offsets within the original context. During preprocessing, each example may be split into multiple overlapping spans, and token-level start/end labels are computed for each span. A4-b changes the order and grouping of examples by bucketing them by length, but our initial implementation reused labels assuming the original ordering. This created “mismatched” pairs of tokenized inputs and answer labels: the model was trained to predict answer spans that did not correspond to the current input span.

Because the loss was still finite and decreased during training, this bug was not immediately obvious from logs alone. Only the final evaluation (very low HasAns F1 and overall F1) revealed the failure. We therefore treat A4-b as a negative ablation that highlights how small changes to batching logic can silently break span-based tasks, and we do not

consider it a valid efficiency improvement. In contrast, A4-nb retains the original example order and only trims padding within each batch, so label alignment remains correct and accuracy is preserved.

C. Practical T4 Training Recipes

Based on our results, we can summarize a few concrete recipes for BERT-base on a 16 GB T4, depending on the constraint:

TABLE IV
SUGGESTED CONFIGURATIONS FOR DIFFERENT T4 CONSTRAINTS.

Scenario	Recommended config	Rationale
Best F1 under fixed GPU budget	A4-nb	Full FT, SDPA + <code>torch.compile</code> + dynamic padding; best F1 and higher throughput than A1/A3.
Tight VRAM, moderate speed constraints	C1	8-bit AdamW cuts optimizer memory; F1 $\approx$ A3, small throughput penalty.
Very tight VRAM	C2	Grad checkpointing reduces activations; $\approx$ 30% VRAM savings vs A3 at lower speed.
Latency-sensitive or quick prototyping	B3	Freezing most encoder layers yields highest throughput with moderate F1 drop.

In other words, for “normal” training on a T4 we recommend A4-nb as the default. When memory is the primary constraint, C1 or C2 are preferable. For rapid experimentation or interactive settings where F1 can be slightly lower, B3 offers the fastest training throughput.

D. Comparison with Previous Work

LoRA and BitFit were originally reported to match or closely track full fine-tuning on several classification and generation tasks using larger models and longer training schedules [1], [2]. In our extractive QA setting with limited epochs, we observe a larger gap, suggesting that these techniques are more sensitive to optimization hyperparameters and task structure than sometimes implied.

Similarly, FlashAttention-style kernels are designed for large sequence lengths and high-end GPUs [3]. Our use of SDPA on T4 shows that fused attention still helps, but the gains are modest because the workload is smaller and constrained by memory and general compute rather than pure attention cost.

The memory savings from 8-bit optimizers agree qualitatively with the findings of Dettmers et al. [4], even though our experiments are limited to a single model and hardware type.

E. Challenges and Limitations

The main limitations of our study are:

- **Hardware constraints:** all core experiments are run on a single T4; A100 experiments remained out of scope due to queue and time limits.
- **Limited hyperparameter tuning:** we use a fixed schedule (3 epochs, mostly fixed learning rates), which likely under-serves LoRA and BitFit.

- **Profiling tools:** cluster restrictions prevented us from running Nsight Systems/Compute, so we could not report SM occupancy or detailed kernel timelines.
- **Single dataset and model:** results may not transfer directly to larger encoders or generative LLMs.

VI. CONCLUSION

We presented a systematic study of fine-tuning `bert-base-uncased` on SQuAD v2 under realistic T4 GPU constraints. By organizing experiments into A-, B-, C-, and D-series, we quantified how system-level optimizations, parameter-efficient methods, and memory-focused techniques interact in practice.

Our main contributions, summarized empirically on SQuAD v2 with BERT-base on a single T4, are:

- SDPA and `torch.compile` yield modest but robust throughput gains without hurting accuracy.
- Dynamic padding (A4-nb) provides the best accuracy–efficiency trade-off among our full fine-tuning configurations.
- Naively implemented bucketing (A4-b) can silently break label alignment and destroy accuracy, highlighting the need for careful batching logic.
- LoRA and BitFit, while attractive for large models, underperform full fine-tuning on SQuAD v2 under our training budget.
- 8-bit optimizers and gradient checkpointing significantly reduce VRAM, enabling more headroom on 16 GB GPUs at acceptable cost in speed and F1.

Future work includes exploring flash-attention kernels on A100-class GPUs, more aggressive LoRA tuning (e.g., higher rank, longer training), and extending the analysis to encoder–decoder and decoder-only LLMs.

ACKNOWLEDGMENT

We thank the NYU HPC support staff for access to GPU resources and course staff for feedback throughout the project. Our code and experiment scripts are available at: <https://github.com/devanshii09/hpml-bert-squadv2>.

REFERENCES

- [1] E. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. ICLR*, 2022.
- [2] E. Ben-Zaken, S. Ravfogel, and Y. Goldberg, “BitFit: Simple Parameter-Efficient Fine-Tuning for Transformer-Based Masked Language Models,” in *Proc. ACL*, 2022.
- [3] T. Dao, D. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Advances in Neural Information Processing Systems*, 2022.
- [4] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, “8-bit Optimizers via Block-wise Quantization,” in *Advances in Neural Information Processing Systems*, 2022.
- [5] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa-V3: Improving DeBERTa and BERT with Knowledge Distillation,” arXiv:2111.09543, 2021.